

Hadoop Random Access Databases

Hbase

Cassandra

CouchDB

Dynamo

MongoDB



Why Hbase ?

- RDBMS get exponentially slow as the data becomes large. It can easily work on extremely large scale data.
- Expected data to be highly structured, i.e. ability to fit in a well-defined schema.
- For both structured and semi-structured data types we can use it.
- For sparse datasets (the data stored for a particular feature contains mostly zeroes), too much of overhead of maintaining NULL values.
- Apache HBase has a distributed architecture.
- HBase offers high security and easy management.
- Moreover, the MapReduce jobs can be backed with HBase Tables.



Master Server

- Assigns regions to the region servers and takes the help of Apache Zookeeper for this task.
- Handles load balancing of the regions across region servers. It unloads the busy servers and shifts the regions to less occupied servers.
- Maintains the state of the cluster by negotiating the load balancing.
- It is responsible for schema changes and other metadata operations such as creation of tables and column families.



Region Server

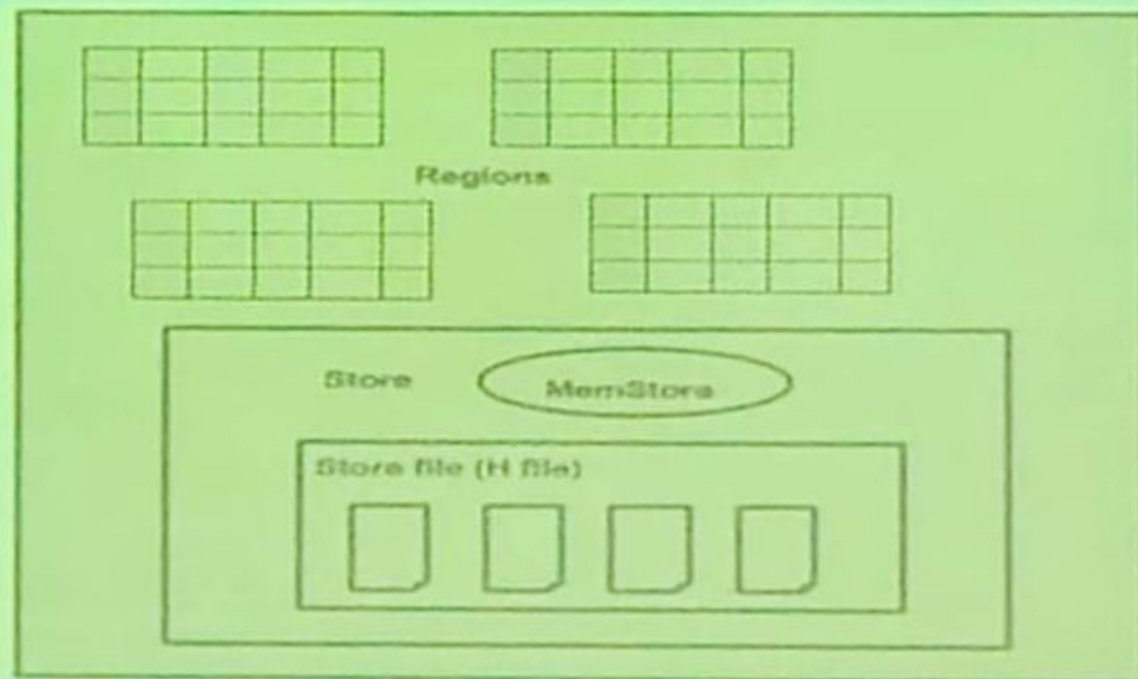
Region-Regions are nothing but tables that are split up and spread across the region servers.

Region server: The region servers have regions that -

- Communicate with the client and handle data-related operations.
- Handle read and write requests for all the regions under it.
- Decide the size of the region by following the region size thresholds.



Region Server



The store contains memory store and HFiles. Memstore is just like a cache memory. Anything that is entered into the HBase is stored here initially. Later, the data is transferred and saved in Hfiles as blocks and the memstore is flushed.



Storage Mechanism in HBase

Rowid	Column Family			Column Family			Column Family			Column Family		
	col1	col2	col3	col1	col2	col3	col1	col2	col3	col1	col2	col3
1												
2												
3												

- Table is a collection of rows.
- Row is a collection of column families.
- Column family is a collection of columns.
- Column is a collection of key value pairs.



Column Oriented V/S Row Oriented

Row Oriented Database	Column Oriented Database
It is suitable for Online Transaction Process (OLTP).	It is suitable for Online Analytical Processing (OLAP).
Such databases are designed for small number of rows and columns.	Column-oriented databases are designed for huge tables.
Record are accessed in sequential manner.	Record are accessed in Random manner.
It used when the size of data less than GB.	It used when the size of data PB or YB.

- Apache HBase is used to have random, real-time read/write access to Big Data.
- It hosts very large tables on top of clusters of commodity hardware.
- Apache HBase is a non-relational database modeled after Google's Bigtable. Bigtable acts up on Google File System, likewise Apache HBase works on top of Hadoop and HDFS.

Column Oriented Database

COLUMN FAMILIES

Row key	personal data		professional data	
	name	city	designation	salary
empid				
1	raju	hyderabad	manager	50,000
2	ravi	chennai	sr.engineer	30,000
3	rajesh	delhi	jr.engineer	25,000



HDFS V/S HBase

HDFS	HBase
HDFS is a distributed file system suitable for storing large files.	HBase is a database built on top of the HDFS.
HDFS does not support fast individual record lookups.	HBase provides fast lookups for larger tables.
It provides high latency batch processing.	It provides low latency access to single rows from billions of records (Random access).
It provides only sequential access of data.	HBase internally uses Hash tables and provides random access, and it stores the data in indexed HDFS files for faster lookups.



Features ?

- HBase is linearly scalable.
- It has automatic failure support.
- It provides consistent read and writes.
- It integrates with Hadoop, both as a source and a destination.
- It has easy java API for client.
- It provides data replication across clusters.
- Horizontally scalable: You can add any number of columns anytime.
- Integrations with Map/Reduce framework: All the commands and java codes internally implements Map/ Reduce to do the task and it is built over Hadoop Distributed File System.



Features.. ?

- Sparse, distributed, persistent, multidimensional sorted map, which is indexed by rowkey, column key, and timestamp.
- Often referred as a key value store or column family-oriented database, or storing versioned maps of maps.
- Fundamentally, it's a platform for storing and retrieving data with random access.
- It doesn't care about data types (storing an integer in one row and a string in another for the same column).
- It is designed to run on a cluster of computers, built using commodity hardware.
- It hosts very large tables on top of clusters of commodity hardware.
- Apache HBase is a non-relational database modeled after Google's Bigtable. Bigtable acts up on Google File System, likewise Apache HBase works on top of Hadoop or HDFS.



Apache HBase

Who ?

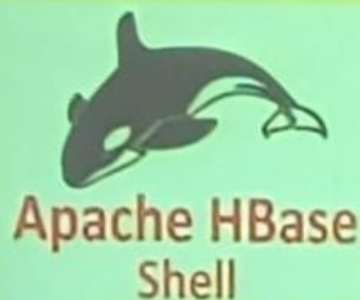
- Pinterest
- Goibibo
- Facebook
- Yahoo
- Twitter
- Adobe

Big Data Analytics



Apache HBase Shell

Presented By
Dr. Yogesh Kumar Gupta



Basics of HBase Shell

- To start the HBase interactive shell :
cloudera@quickstart ~>hbase shell

It will Launch the shell as-

hbase(main):001:0>

- To check for the list of supported commands.

hbase(main):001:0>help

- To leave the HBase Shell

hbase(main):001:0>exit

Or you can use <ctrl+c>.

- To list all the tables in Hbase

hbase(main):001:0>list



Data Definition Language Commands

These are the commands that operate on the tables in HBase.

- 1) **Create**
- 2) **list**
- 3) **Disable**
- 4) **is_disabled**
- 5) **Enable**
- 6) **is_enabled**
- 7) **Describe**
- 8) **Alter**
- 9) **exists**
- 10) **Drop**



Apache HBase
Shell

Data Definition Language Commands

Create a Table

Syntax : create '<table name>', '<column family>'

Example:

```
hbase(main):001:0> create 'emp', 'personal data', 'professional data'
```

```
0 row(s) in 1.1300 seconds
```

```
=> Hbase::Table - emp
```

Row Key	Personal Data	Professional Data
---------	---------------	-------------------

Listing a Table

List : To list all the tables in Hbase.

Example:

```
hbase(main):001:0> list
```

```
TABLE
```

```
emp
```



Apache HBase
Shell

Data Definition Language Commands

Disabling a Table

disable_all : To disable all the tables matching the given regex.

Example:

```
hbase(main):001:0> disable_all 's.*'
```

```
student
```

```
services
```

```
Disable the above 2 tables (y/n)? Y
```

```
2 tables successfully disabled
```

Scanning the data in Htable

Example:

```
hbase(main):001:0> scan 'student'
```

```
ROW COLUMN + CELL
```

```
ERROR: student is disabled.
```



Apache HBase
Shell

Data Definition Language Commands

Disabling a Table

Disable : To disable a table.

Example:

```
hbase(main):001:0> disable 'emp'
```

Output:

```
0 row(s) in 1.2760 seconds
```

is_disabled : To verifies whether the table named emp is disabled or not.

Example:

```
hbase(main):001:0> is_disabled 'emp'
```

Output:

```
True
```

```
0 row(s) in 0.0440 seconds
```



Apache HBase
Shell

Data Definition Language Commands

Enabling a Table

Enable: To enable a table.

Example:

```
hbase(main):001:0> enable 'emp'
```

Output:

```
0 row(s) in 0.4580 seconds
```

is_enabled: To verify whether the table named emp is enabled or not.

Example:

```
hbase(main):001:0> is_enabled 'emp'
```

Output:

```
True
```

```
0 row(s) in 0.0440 seconds
```



Data Definition Language Commands

Altering an existing Table

Alter: to make changes to an existing table

1. maximum number of cells of a column family
2. set and delete table scope operators
3. Add or delete a column family to and from a table.

1. Changing the Maximum Number of Cells of a Column Family

```
hbase> alter 't1', NAME => 'f1', VERSIONS => 5
```

Example:

```
hbase(main):001:0> alter 'emp', NAME => 'personal data', VERSIONS => 5
```

Updating all regions with the new schema...

0/1 regions updated.

1/1 regions updated.

Done.

0 row(s) in 2.3050 seconds



Data Definition Language Commands

Altering an existing Table

2. set and delete table scope operators:

table scope operators are: -

READONLY.

MAX_FILESIZE.

MEMSTORE_FLUSH_SIZE.

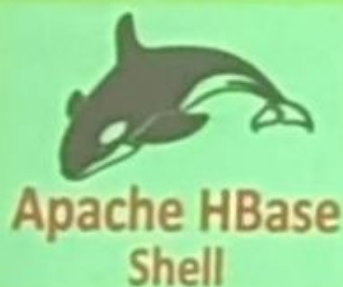
DEFERRED_LOG_FLUSH.

Setting READONLY

```
hbase> alter 't1', READONLY(option)
```

Example:

```
hbase(main):001:0> alter 'emp', READONLY=>'true'
```



Data Definition Language Commands

Altering an existing Table

Setting MAX_FILESIZE.

```
hbase(main):001:0> alter 'emp', MAX_FILESIZE => '12345678'
```

Setting MEMSTORE_FLUSH_SIZE.

```
hbase(main):001:0> alter 'emp', MEMSTORE_FLUSH_SIZE  
=> '12345678'
```

Setting DEFERRED_LOG_FLUSH.

```
hbase(main):001:0> alter 'emp', DEFERRED_LOG_FLUSH => 'true'
```

Removing Table Scope Operators

```
hbase> alter 't1', METHOD => 'table_att_unset', NAME  
=> 'MAX_FILESIZE'
```



Data Definition Language Commands

Deleting a Column Family

```
hbase> alter 'table name', 'delete' => 'column family'
```

```
hbase(main):001:0> scan 'employee'
```

ROW	COLUMN+CELL
-----	-------------

row1	column = personal:name, timestamp = 1418193806767, value = raju
------	---

row1	column = professional:designation, timestamp = 1418193767, value = manager
------	--

1 row(s) in 0.0160 seconds

Now let us delete the column family named **professional** using alter command.

```
hbase(main):001:0> alter 'employee', 'delete' => 'professional'
```



Apache HBase
Shell

Data Definition Language Commands

Existence of Table

Exists : To verify the existence of a table.

```
hbase(main):001:0> exists 'emp'
```

Output:

Table emp does exist

0 row(s) in 0.0750 seconds

Describing a Table

describe: To describe the table.

```
hbase(main):001:0> describe 'emp'
```



Apache HBase
Shell

Data Definition Language Commands

Dropping a Table

drop: To delete a table.

```
hbase(main):001:0> disable 'emp'
```

```
0 row(s) in 1.4580 seconds
```

```
hbase(main):001:0> drop 'emp'
```

```
0 row(s) in 0.3060 seconds
```

```
hbase(main):001:0> exists 'emp'
```

```
Table emp does not exist
```

```
0 row(s) in 0.0730 seconds
```



Data Definition Language Commands

Dropping a Table

Drop_all: To drop the table matching the “regex”.

```
hbase(main):001:0> disable_all 'stu.*'
```

```
student
```

```
stu_course
```

```
stu_class
```

```
Disable the above 3 tables (y/n)?y
```

```
3 tables successfully disabled
```

```
hbase(main):001:0> drop_all 'stu.*'
```

```
student
```

```
stu_course
```

```
stu_class
```

```
Drop the above 3 tables (y/n)?y
```

```
3 tables successfully dropped
```



Data Manipulation Language Commands

DML commands are use to perform the operations on data.

- 1) Put
- 2) Get
- 3) Delete
- 4) Deleteall
- 5) scan
- 6) Count
- 7) truncate



Data Manipulation Language Commands

Inserting Data into Table

put - Puts a cell value at a specified column in a specified row in a table.

Syntax :

```
put '<table name>','row1','<colfamily:colname>','<value>'
```

Example:

Inserting the First Row

```
hbase(main):005:0> put 'emp','1','personal data:name','ravi'
```

```
0 row(s) in 0.6600 seconds
```

```
hbase(main):006:0> put 'emp','1','personal data:city','jaipur'
```

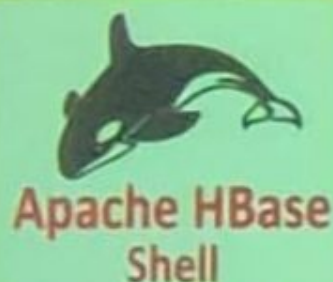
```
0 row(s) in 0.0410 seconds
```

```
hbase(main):007:0> put 'emp','1','professional data:designation','manager'
```

```
0 row(s) in 0.0240 seconds
```

```
hbase(main):007:0> put 'emp','1','professional data:salary','50000'
```

```
0 row(s) in 0.0240 seconds
```



Data Manipulation Language Commands

Verify the Data of a Table

```
hbase(main):022:0> scan 'emp'
```

ROW COLUMN+CELL

1 column=personal data:city, timestamp=1417524216501, value= jaipur

1 column=personal data:name, timestamp=1417524185058, value=ravi

1 column=professional data:designation, timestamp=1417524232601, value=manager

1 column=professional data:salary, timestamp=1417524244109, value=50000

Row key	Personal Data		Professional Data	
Empid	Name	City	Designation	Salary
1	Ravi	Jaipur	Manager	50000



Data Manipulation Language Commands

Updating Data into Table

Syntax :

```
put '<table name>','row1','<colfamily:colname>','<value>'
```

Example:

```
hbase(main):002:0> put 'emp','row1','personal:city','Delhi'
```

0 row(s) in 0.0400 seconds

Row key		Personal Data		Professional Data	
Empid		Name	City	Designation	Salary
1		Ravi	Delhi	Manager	50000



Data Manipulation Language Commands

Reading Data from a Table

Syntax :

```
get '<table name>', 'row1'
```

Example:

```
hbase(main):012:0> get 'emp', '1'
```

COLUMN	CELL
--------	------

personal : city timestamp = 1417521848375, value = delhi
--

personal : name timestamp = 1417521785385, value = ravi

professional: designation timestamp = 1417521885277, value = manager
--

professional: salary timestamp = 1417521903862, value = 50000

4 row(s) in 0.0270 seconds



Data Manipulation Language Commands

Reading a Specific Column from Table

Syntax :

get 'table name', 'rowid', {COLUMN $\Rightarrow$ 'column family:column name'}

Example:

```
hbase(main):015:0> get 'emp', 'row1', {COLUMN  $\Rightarrow$  'personal:name'}
```

COLUMN	CELL
--------	------

personal : name	timestamp = 1417521785385, value = ravi
-----------------	---

1 row(s) in 0.0080 seconds



Apache HBase
Shell

Data Manipulation Language Commands

Versioning in Hbase

Versioning is just like Hadoop replication factor. In Hadoop by default Replication factor is 3. In HBase Versioning default is 3 and you can change it as per your requirement. In other words Versioning is nothing but maintaining the history of a column.

```
alter 'emp', {NAME=>'professional data',VERSIONS=>4}
```

```
hbase(main):212:0> put 'emp', 1, 'professional data:salary', 6000
```

```
0 row(s) in 0.0330 seconds
```

```
hbase(main):213:0> put 'emp', 1, 'professional data:salary', 7000
```

```
0 row(s) in 0.0230 seconds
```

```
hbase(main):214:0> put 'emp', 1, 'professional data:salary', 8000
```

```
0 row(s) in 0.0240 seconds
```

```
hbase(main):215:0> put 'emp', 1, 'professional data:salary', 9000
```

```
0 row(s) in 0.0220 seconds
```

```
hbase(main):216:0> put 'emp', 1, 'professional data:salary', 10000
```

```
0 row(s) in 0.0210 seconds
```



Data Manipulation Language Commands

Versioning in Hbase

when you execute get tablename then it will show the always latest 4 version data.

```
get 'emp', 1, {COLUMN=>' professional data:salary ',VERSIONS => 4 }
```

COLUMN CELL

professional data:salary timestamp=1586586460772, value=10000

professional data:salary timestamp=1586586457185, value=9000

professional data:salary timestamp=1586586454992, value=8000

professional data:salary timestamp=1586586452822, value=7000

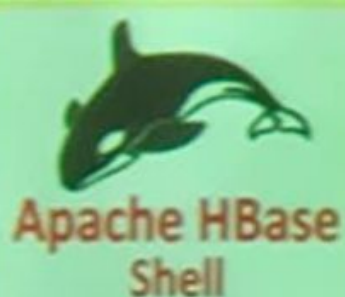
4 row(s) in 0.0580 seconds

And when you execute scan tablename then it will show the always latest version data.

```
hbase(main):218:0* scan 'emp'
```

ROW COLUMN+CELL

1 column= professional data:salary, timestamp=1586586460772, value=10000



Data Manipulation Language Commands

Deleting a Specific Cell in a Table

Syntax :

`delete '<table name>', '<row>', '<column name >'`

Example:

```
hbase(main):006:0> delete 'emp', '1', 'personal data:city'  
row(s) in 0.0060 seconds
```

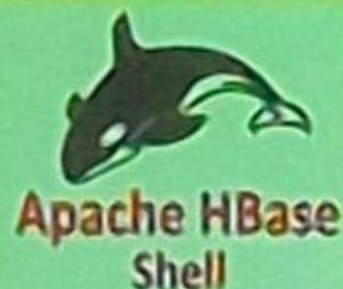
Deleting All Cells in a Table

Syntax :

`deleteall '<table name>', '<row>'`

Example:

```
hbase(main):007:0> deleteall 'emp', '1'  
0 row(s) in 0.0240 seconds
```



Data Manipulation Language Commands

Scanning Table

Syntax :

scan '<table name>'

Example:

hbase(main):010:0> scan 'emp'

ROW COLUMN+CELL

1 column = personal data:city, timestamp = 1417521848375, value = delhi

1 column = personal data:name, timestamp = 1417521785385, value = ravi

1 column = professional data:designation, timestamp = 1417585277, value = manager

1 column = professional data:salary, timestamp = 1417521903862, value = 50000

1 row(s) in 0.0370 seconds

Count: To count rows in a table

Syntax :

count '<table name>'

Example:

hbase(main):023:0> count 'emp'

2 row(s) in 0.090 seconds => 2



Data Manipulation Language Commands

Truncate

This command disables drops and recreates a table.

Syntax :

```
hbase> truncate 'table name'
```

Example:

```
hbase(main):011:0> truncate 'emp'
```

Truncating 'one' table (it may take a while):

Disabling table...

Truncating table...

0 row(s) in 1.5950 seconds

After truncating the table, use the scan command to verify. You will get a **table with zero rows**.

```
hbase(main):017:0> scan 'emp'
```

ROW COLUMN + CELL

0 row(s) in 0.3110 seconds

Value filter: This filter is used to filter based on col. value. It takes an operator (equal, >, ≠, etc) and a byte[] comparator for the cell value.

```
scan 'table-name', { FILTER => " ValueFilter  
    (=, 'binary: v1') " }
```

ValueFilter(=, 'binary: v1'): This filter checks whether the value of a column equals v1. The binary prefix indicates that the value is a binary value. (a byte array) which is how HBase stores data internally.

PrefixFilter: This filter takes one Argument as a prefix of a row Key. It returns those Key-values present in the row that starts with the specified row prefix.

```
scan 'your-table-name', { FILTER =>  
    " PrefixFilter('row1') " }
```

SingleColumnValueFilter: takes a column family, a qualifier, a comparison operator and a comparator. If the specified col. is not found, all the col. of that row will be emitted. If the col. is found and the comparison with the comparator returns true, all the col. of the rows will be emitted.

```
scan 'your-table-name', { FILTER => " SingleColumn  
    ValueFilter('cf', 'col1', '=', 'binary: v1') " }
```